

操作系统作业1

202440012028 陈新安

1. 在Linux系统中，当一个进程调用 `fork()` 创建子进程后，父进程和子进程的PID、PPID、文件描述符、内存空间分别有什么变化？如果父进程在 `fork()` 前打开了一个文件，子进程对该文件的读写操作会对父进程产生什么影响？请结合内核的实现机制分析原因。

当一个进程调用 `fork()` 创建子进程后：

- 父进程的PID，PPID，FD，内存空间都没有发生改变
- 子进程分配到一个唯一的PID，PPID为父进程的PID，FD为父进程FD表的副本，内存空间也为父进程的副本

若父进程在 `fork()` 前打开了一个文件：

在xv6中，调用fork函数时，调用了 `uvmcopy` 函数

```
...
// Copy user memory from parent to child.
if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
    freeproc(np);
    release(&np->lock);
    return -1;
}
...
```

在调用 `uvmcopy` 函数时：

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    ...
    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        ...
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == 0)
            goto err;
        memmove(mem, (char*)pa, PGSIZE);
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
            kfree(mem);
            goto err;
        }
    }
    ...
}
```

`uvmcopy` 函数会将父进程的内存空间复制到子进程中，因此子进程会拥有父进程打开文件的文件描述符的副本。由于文件描述符是指向内核中的文件表项的指针，因此父子进程共享同一个文件表项。

在Linux中，类似的操作由 `copy_on_write` 机制实现，区别在于其只有在父子进程对同一内存页进行写操作时才会真正复制内存页。

也就是说，父进程和子进程无论哪个进程对文件进行了读写操作，因为它们共享同一个文件表项，所以对文件的读写操作会相互影响。比如当父进程对一个文件执行了写操作，当子进程也想进行写操作时，其会从父进程执行写操作结束的地方继续写入，而不是从文件的开头开始写入。

2. 请分析进程的三种基本状态（就绪、运行、阻塞）之间的转换条件，以及在Linux系统中，进程还有哪些扩展状态？这些扩展状态分别对应什么场景？

三种基本状态的转换条件：

- 处于就绪态的程序，当调度程序为其分配处理机后，则进入运行态
- 处于运行态的程序，当分配给其的时间片用完后，则进入就绪态
- 处于运行态的程序，当需要等待某个事件（如I/O请求）时，则进入阻塞态
- 处于阻塞态的程序，若等待的事件发生（如I/O完成）后，则进入就绪态

在Linux系统中，进程有以下扩展状态：

- i. 可中断睡眠状态：进程在等待某个事件时进入可中断睡眠状态，此时该进程可以被信号中断，若接收到信号后，进程会被唤醒并处理信号。
- ii. 不可中断睡眠状态：处理复杂的硬件I/O时，为防止硬件数据传输中断，进程进入不可中断睡眠状态，此时该进程无法被信号中断，必须等到I/O完成后才能继续执行。
- iii. 僵尸状态：当 fork 后生成的子进程已执行 exit 但父进程未调用 wait 回收子进程时，子进程进入僵尸状态，此时子进程的PCB仍然存在于内核中，但其资源已被释放，等待父进程回收。
- iv. 暂停状态：在进程访问软件资源的时候，可能不希望进程进行访问，就发出 sigstop 信号将进程暂停，此时该进程进入暂停状态，直到接收到 sigcont 信号后才会继续执行。

3. 什么是进程的上下文切换？上下文切换的开销主要包括哪些部分？请比较进程上下文切换和线程上下文切换的区别，以及如何减少上下文切换的开销？

进程的上下文切换是指：在处理程序并发执行，进行进程切换时，操作系统保留当前进程的CPU环境，并设置新选中的进程的CPU环境。

上下文切换的开销主要包括以下部分：

- i. CPU状态的保存和恢复：操作系统需要将寄存器的内容，PC，栈指针等信息存储到当前进程的PCB中，并从新选中的进程的PCB中恢复这些信息。
- ii. 内存管理开销：如果新选中的进程不在内存中，操作系统需要进行页面调度，将新进程的代码和数据加载到内存中，这会增加上下文切换的开销。
- iii. cache 失效：上下文切换可能导致CPU中 cache 失效，需要从内存中重新加载数据。
- iv. 调度开销：操作系统用何种调度算法来选择下一个运行的进程也会影响上下文切换的开销。

进程上下文切换和线程上下文切换的区别：

进程上下文切换涉及到整个进程的状态保存和恢复，包括内存空间，文件描述符等资源的切换；而线程上下文切换只涉及到线程的寄存器状态、栈指针等同一类线程中每个线程私有的内容的切换，因为线程共享同一进程的内存空间和资源。

减少上下文切换的开销的方法：

- i. 优化调度算法，尽可能降低上下文切换的频率
- ii. 尽可能减少进程或线程的数量

4. 在Linux系统中，wait() 和 waitpid() 函数的区别是什么？如果父进程不调用 wait() 或 waitpid() 回收子进程，会出现什么问题？如何解决这个问题？请结合僵尸进程和孤儿进程的概念分析。

在Linux V0.01中，wait() 函数的源代码如下：

```
pid_t wait(int * wait_stat)
{
    return waitpid(-1,wait_stat,0);
}
```

可以看到，wait() 函数实际上是 waitpid() 函数的一个特例。

wait 和 waitpid 函数的区别：

- i. wait() 函数会等待任意一个子进程结束，而 waitpid() 函数可以指定等待特定的子进程结束。
- ii. 在第二个参数位上，wait() 函数会将子进程的退出状态存储在 wait_stat 指向的内存中，而 waitpid() 函数可以通过第三个参数来指定是否需要获取子进程的退出状态。

- iii. `waitpid()` 函数还可以通过第三个参数来指定是否需要非阻塞等待子进程结束。

由此可见，如果父进程不调用 `wait()` 或 `waitpid()` 回收子进程，会导致子进程进入僵尸状态，即子进程已经执行完毕但其父进程尚未对其进行回收。此时子进程的PCB仍然存在于内核中，其资源也已被释放，等待父进程回收。要解决这个问题，只需要调用 `wait()` 或 `waitpid()` 函数来回收子进程即可。

5. 什么是进程间通信（IPC）？请比较管道、命名管道、消息队列、共享内存、套接字这几种IPC机制的优缺点和适用场景。并分析为什么共享内存是效率最高的IPC机制？

进程间通信指的是：不同进程之间的内存和数据都是独立的。如果想要在不同进程间进行数据的交换，就需要使用进程间通信机制。

IPC机制	优点	缺点	适用场景
管道	简单，轻便	只能在父子进程之间使用；单向通信	父子进程之间的简单数据交换
命名管道	支持无亲缘关系的进程之间通信	单向通信	无亲缘关系的进程之间的简单数据交换
消息队列	支持消息的优先级；适合复杂的数据交换	实现复杂；开销较大	需要复杂数据交换和优先级处理的场景
共享内存	不需要内核中转，速度最快	需要同步机制来保证数据一致性	需要高性能数据交换的场景
套接字	支持网络通信；跨平台	实现复杂；开销较大	分布式系统等需要网络通信的场景

为什么共享内存是效率最高的IPC机制：

共享内存机制允许多个进程直接访问同一块内存区域，不需要通过内核进行数据的复制和传输，因此具有最低的通信开销和最高的效率。相比之下，其他IPC机制如管道、消息队列和套接字都需要内核介入来传递数据，这会增加额外的开销和延迟。

6. 在Linux系统中，匿名管道和命名管道的核心区别是什么？如果需要在两个无亲缘关系的进程之间实现双向通信，应该选择哪种IPC机制？请结合内核实现原理分析原因，并给出具体的代码实现思路。

在Linux系统中，两种管道的声明如下：

```
int mkfifo ( const char * _path , mode_t mode );
int pipe ( int * fildes );
```

两者的核心区别在于：匿名管道只能在父子进程之间使用，而命名管道可以在无亲缘关系的进程之间使用。

如果要在两个无亲缘关系的进程之间实现双向通信，应使用命名管道，且因为管道只支持单向通信，需要创建两个命名管道来实现双向通信。

```
mkfifo("fifo1", 0666); // 进程A向进程B发送数据
mkfifo("fifo2", 0666); // 进程B向进程A发送数据

// 进程A
int fd1 = open("fifo1", O_WRONLY);
int fd2 = open("fifo2", O_RDONLY);
// 进程A向进程B发送数据
write(fd1, 'A', 1);
// 进程A从进程B接收数据
char buffer[100];
read(fd2, buffer, sizeof(buffer));

// 进程B
int fd1 = open("fifo1", O_RDONLY);
int fd2 = open("fifo2", O_WRONLY);
// 进程B从进程A接收数据
char buffer[100];
read(fd1, buffer, sizeof(buffer));
// 进程B向进程A发送数据
write(fd2, 'B', 1);
```

7. 在Linux系统中，进程的内存空间包括哪些部分？请分别说明每个部分的作用，以及在进程创建和运行过程中，这些部分是如何变化的？结合 `fork()` 和 `exec()` 函数分析内存空间的变化。

Linux系统中，进程的内存空间从下到上主要包括以下几个部分：
文本区，数据区，bss区，堆区，栈区和内核空间。

```
// kernel/fork.c
int copy_mem(int nr, struct task_struct * p)
{
    ...
    new_data_base = new_code_base = nr * 0x4000000;
    set_base(p->ldt[1], new_code_base);
    set_base(p->ldt[2], new_data_base);
    if (copy_page_tables(old_data_base, new_data_base, data_limit)) {
        free_page_tables(new_data_base, data_limit);
        return -ENOMEM;
    }
    return 0;
}

int copy_process(int nr, ...)
{
    ...
    if (copy_mem(nr, p)) {
        free_page((long) p);
        return -EAGAIN;
    }
    for (i=0; i<NR_OPEN; i++)
        if (f=p->filp[i])
            f->f_count++;
    if (current->pwd)
        current->pwd->i_count++;
    if (current->root)
        current->root->i_count++;
    set_tss_desc(gdt+(nr<<1)+FIRST_TSS_ENTRY, &(p->tss));
    set_ldt_desc(gdt+(nr<<1)+FIRST_LDT_ENTRY, &(p->ldt));
    ...
}

// kernel/exec.c
int do_execve(unsigned long * eip, long tmp, char * filename,
              char ** argv, char ** envp)
{
    ...
    /* OK, This is the point of no return */
    for (i=0; i<32; i++)
        current->sig_fn[i] = NULL;
    for (i=0; i<NR_OPEN; i++)
        if ((current->close_on_exec>>i)&1)
            sys_close(i);
    current->close_on_exec = 0;
    free_page_tables(get_base(current->ldt[1]), get_limit(0x0f));
    free_page_tables(get_base(current->ldt[2]), get_limit(0x17));
    if (last_task_used_math == current)
        last_task_used_math = NULL;
    current->used_math = 0;
    p += change_ldt(ex.a_text, page) - MAX_ARG_PAGES * PAGE_SIZE;
    p = (unsigned long) create_tables((char *)p, argc, envc);
    current->brk = ex.a_bss +
        (current->end_data = ex.a_data +
         (current->end_code = ex.a_text));
    current->start_stack = p & 0xfffff000;
    ...
}
```

各部分的作用：

- 文本区： 存放程序的机器指令代码，只读
- 数据区： 存放程序中已初始化的全局变量和静态变量
- bss区： 存放程序中未初始化的全局变量和静态
- 堆区： 存放动态分配的内存
- 栈区： 存放函数调用时的参数，返回地址和局部变量等
- 内核空间： 存放内核代码和数据，用户进程无法访问

在进程创建和运行过程中，这些部分的变化如下：

当调用 `fork()` 函数创建子进程时，子进程会复制父进程的内存空间，包括文本区、数据区、bss区、堆区和栈区。子进程和父进程共享同一内核空间。

调用 `exec()` 函数执行一个新的程序时，当前进程的内存空间会被新的程序替换，文本区、数据区、bss区、堆区和栈区等都会被新的程序重新分配和初始化。内核空间仍然保持不变。

在运行过程中，进程的内存空间可能会发生变化，例如堆区的动态内存分配和栈区的函数调用等都会导致内存空间的变化。

当进程结束时，其内存空间会被操作系统回收，释放占用的资源。

8. 什么是进程的优先级?Linux系统中如何表示和修改进程的优先级?请分析实时进程和普通进程在调度上的区别，以及实时进程的优先级范围。

Linux进程中优先级决定了哪个进程会优先获得CPU资源。

Linux系统中，进程的优先级分为普通优先级和实时优先级：

- 普通优先级用 `nice` 值表示，范围从-20 到 19，值越小优先级越高。用户可用 `nice` 命令来修改进程的 `nice` 值从而修改进程的优先级。
- 实时优先级用调度策略和优先级值表示，优先级值范围从1到99，值越大优先级越高。使用 `root + SCHED_FIFO / SCHED_RR` 调度策略。用户可用 `chrt` 命令来修改进程的调度策略和优先级。

两种优先级的区别在于：

普通进程采用完全公平调度算法，使得各个程序尽量公平地获得CPU资源；

实时进程采用实时调度算法，保证高优先级的实时进程能够优先获得CPU资源，以满足实时任务的需求。

9. 比较消息队列、信号量和套接字在进程间通信中的优缺点和适用场景。如果需要一个支持多进程并发访问的日志系统，应该选择哪种IPC机制?请给出详细的设计方案。

IPC机制	优点	缺点	适用场景
消息队列	支持信息的异步传输，通信双方无需同时交互	实现复杂 开销大	在微服务和无服务器基础架构中实现应用程序到应用程序的通信
信号量	适合同步和互斥；实现简单	不适合传递大量数据	需要同步和互斥的场景
套接字	支持网络通信；跨平台	实现复杂；开销较大	分布式系统等需要网络通信的场景

如果来实现一个支持多并发访问的日志系统，应该使用消息队列。

10. 在Linux系统中，`vfork()` 和 `fork()` 函数的区别是什么? `vfork()` 的设计初衷是什么?为什么现在 `vfork()` 的使用场景越来越少?请结合写时复制（Copy-On-Write）机制分析。

`vfork()` 和 `fork()` 函数的区别在于：

执行 `vfork()` 后，会保证子进程先执行，只有在其执行 `exec()` 或 `exit()` 后，父进程才会继续执行；而 `fork()` 函数则没有这样的保证，父子进程的执行顺序是不确定的，需要手动 `sleep` 。

同时，`vfork()` 函数不会复制父进程的内存空间，而是让子进程共享父进程的内存空间，直到子进程调用 `exec()` 或 `exit()` 。而 `fork()` 函数会复制父进程的内存空间给子进程。

随着 `cow` 机制的引入，`fork()` 函数只在子进程需要修改内存时才会复制内存页，一定程度上代替了 `vfork()` 的作用，故而现在 `vfork()` 的使用场景越来越少。

11. 共享内存是最快的IPC机制，但为什么实际使用中很少单独使用？请分析共享内存与信号量、消息队列配合使用的典型场景，并说明如何通过信号量解决共享内存的同步与互斥问题。

很少单独使用的原因：

共享内存使程序直接操作物理内存，当多进程并发访问内存时，同步机制的缺失会导致写入和读取数据不一致的问题。

典型场景：

共享内存：高频大数据量传输，实时系统，数据库内核

信号量：共享资源的保护，进程同步，分布式系统中的分布式锁

消息队列：用于在进程之间传递消息，通知其他进程共享内存中的数据已经更新或需要处理。

通过信号量解决共享内存的同步与互斥问题：

信号量是用于进程间同步与互斥的计数器，其通过维护一个整数，通过 `p()` 和 `v()` 操作来实现对共享资源的访问控制。信号量初始化为1，表示资源可用。当一个进程需要访问共享内存时，执行 `p()` 操作将信号量减1。若执行 `p()` 操作后信号量的值为0，说明当前进程可以使用共享内存，而其他进程需要等待。当进程访问完共享内存后，执行 `v()` 操作将信号量加1，表示资源可用，唤醒等待的进程。

通过维护信号量，既保证了共享内存的互斥访问，同一时间内只有一个进程访问共享内存；也保证了多个进程的执行顺序。

12. 在Linux系统中，`ftok()` 函数生成的键值唯一吗？如果两个进程使用不同的文件路径或项目ID调用 `ftok()` 函数，会生成相同的键值吗？请分析 `ftok()` 函数的实现原理，并说明如何避免键值冲突。

`ftok()` 函数生成的键值不唯一，因为其生成方式是通过设备号，`inode`，以及项目ID进行位截断并拼接而成。

由于 `ftok()` 函数只取了文件的 `inode` 号的低16位和项目ID的低8位，因此就算两个进程使用不同的文件路径或项目ID调用 `ftok()` 函数，只要分配的 `inode` 和 `Id` 相差足够大，就会产生相同的键值。

`ftok()` 通过以下方式生成键值：

$$Key = ((st_{dev} \& 0xff) \ll 24) | ((st_{ino} \& 0xffff) \ll 16) | (proj_{id} \& 0xff)$$

其中：

- `stdev`：文件所在设备的设备号
- `stino`：文件的 `inode` 号
- `projid`：用户指定的项目ID

规避键值冲突的方法有：

- 人工指定一个唯一的项目ID，确保不同进程使用不同的项目ID
- 选择不同的文件路径，确保生成的 `inode` 号及设备号不同
- 使用 `IPC_PRIVATE` 作为键值，确保每个 `IPC` 通道都是唯一的

13. 在Linux系统中，如何使用 `mmap()` 函数实现进程间通信？与共享内存相比，`mmap()` 有什么优势和劣势？请给出具体的代码实现示例。

`mmap()` 函数与共享内存机制类似，都是通过映射内存来实现进程间通信的。

优点：

- 避免 `read` 和 `write` 系统调用的开销，直接在用户空间进行数据交换，更加高效
-

缺点：

- 映射长度不宜过大，否则效率较低
- 无法像共享内存那样实现高频大数据量的传输

实现示例：大文件局部批量修改

```
...
int main() {
    const char *filename = "biglog.log";
    int fd = open(filename, O_RDWR);
    struct stat sb;
    fstat(fd, &sb);
    size_t filesize = sb.st_size;

    void *map = mmap(NULL, filesize, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) { perror("mmap"); return 1; }

    // 假设在偏移100000处替换为"ERROR"
    memcpy((char*)map + 100000, "ERROR", 5);

    msync(map, filesize, MS_SYNC); // 强制刷回磁盘
    munmap(map, filesize);
    close(fd);
    return 0;
}
```

14. 请解释线程控制块（TCB）的主要作用和包含的核心字段？对比进程控制块（PCB），说明TCB在设计上有哪些简化和差异？

主要作用：

记录线程的运行状态，保存线程的上下文信息，管理线程的调度和资源分配，保证线程运行的独立性。

核心字段：

ID、状态、寄存器信息、PC、优先级、栈指针等

简化：

- 同一类线程共享同一进程的内存空间和资源，因此TCB不需要记录内存空间和文件描述符等信息
- 线程的调度和资源分配相对于进程更简单，因此TCB的设计也相对简化
- TCB的设计也更轻量级，线程的切换比进程更快

15. 请分析用户级线程（ULT）和内核级线程（KLT）的核心区别，以及在多CPU系统中，用户级线程的并发执行能力为什么会受到限制？如果一个进程包含多个用户级线程，当其中一个线程执行阻塞系统调用时，其他用户级线程会受到什么影响？请结合内核调度机制分析原因。

核心区别：

ULT在用户空间中实现，其对线程的创建、撤销、同步和通信等功能都无需内核支持，即ULT与内核无关

受限制的原因：

内核每次分配给一个进程仅有一个CPU时间片，因此即使一个进程包含多个用户级线程，该进程在任意时刻只能有一个线程在CPU上运行。

在基于进程机制的操作系统中，系统调用的阻塞是针对整个进程的，因此，当线程执行一个系统调用时，不仅该线程会被阻塞，而且进程内的所有线程都会被阻塞，而在KST方式下，进程中的其他线程仍然可以继续运行。

16. 在多线程环境中，什么是“重排序”？重排序会带来什么问题？volatile关键字是如何禁止重排序的？请结合Java内存模型（JMM）分析原因。

重排序指的是：在不影响单线程程序执行结果的前提下，编译器或处理器为了优化性能而改变指令的执行顺序。

问题：代码执行顺序的该变可能导致数据异常或程序崩溃，如异常的对象实例初始化

使用 volatile 关键字时，JMM 会在对应的指令插入内存屏障（loadload、storestore、loadstore、storeload），保证读或写操作间的顺序性，从而禁止重排序。

17. 什么是线程的“上下文切换”?线程上下文切换的开销主要包括哪些部分?如何减少线程上下文切换的开销?请比较进程上下文切换和线程上下文切换的差异，并分析为什么线程上下文切换的开销比进程小。

线程的上下文切换指的是：
在操作系统处理多线程并发时，保留当前线程的CPU环境，加载新选中的线程的CPU环境，执行新线程的过程。

开销：寄存器状态的保存和恢复、 cache 失效、调度开销、内核态切换开销等

如何减少线程上下文切换的开销：
优化调度算法，减小上下文切换的频率；减少线程的数量

差异：
线程上下文切换只涉及线程的寄存器状态、栈指针等同一类线程中每个线程私有的内容的切换，因为线程共享同一进程的内存空间和资源；而进程上下文切换涉及到整个进程的状态保存和恢复，包括内存空间，文件描述符等资源的切换。

18. 在Linux系统中，进程的内存空间包括哪些部分?请分别说明每个部分的作用，以及在进程创建和运行过程中，这些部分是如何变化的?结合 fork() 和 exec() 函数分析内存空间的变化。

同第七题 略

19. 请对比“一对一”、“一对多”和“多对多”三种线程映射模型的实现原理和适用场景?在Linux系统中，采用的是哪种线程映射模型?为什么这种模型更适合现代操作系统？

线程映射模型	实现原理	适用场景
一对一	每个用户级线程对应一个内核级线程	需要高并发和多CPU支持的场景
一对多	多个用户级线程对应一个内核级线程	资源受限的环境，如嵌入式系统
多对多	多个用户级线程对应多个内核级线程	需要高并发和多CPU支持的场景，同时需要更好的资源利用率

Linux系统采用的是一对一线程映射模型，因为其有以下优点：

- 采用一对多或多对多模型时，当某个线程进入阻塞态，会导致整个内核线程被阻塞，影响其他线程的执行；而一对一模型中，每个用户级线程对应一个内核级线程，某个线程进入阻塞态时，不会影响其他线程的执行。
- 一对一模型中，线程调度和管理相对简单。
- 可将一个进程下的线程分布在多个CPU上执行，充分利用多核处理器的性能。

20. 在用户级线程库中，如何实现线程的调度?请举例说明一种用户级线程调度算法，并分析其优缺点。为什么用户级线程调度不需要内核的参与？

如何实现线程调度：
通过维护一个TCB链表来实现调度

用户级线程调度算法：轮转调度算法
实现方式：维护一个就绪队列，每次调度时将当前线程放到队列末尾，选择队列头部的线程执行。
优点：简单，切换过程只需要进行一次上下文切换；公平；完全在用户态实现，不需要内核的参与
缺点：无法处理优先级；当一个线程进入阻塞态时，整个进程的所有线程都会被阻塞；无法利用多核处理器的性能

用户级线程调度不需要内核参与的原因：
线程共享同一进程的内存空间和资源，所以只需要修改PC等线程私有的内容即可实现线程切换，无需内核介入